

From Pixels to a Periodic Phase Signal: Per-Shot Kinematic Inference and Its Effect on (A_s, A_c) Recovery

Contents

1	Problem Statement	2
2	The Reduction: From (A_s, A_c) to Per-Shot θ_{iz}	2
2.1	The observation model	2
2.2	Step 1: the nuisance phase is a per-shot, per-plane-pair quantity	2
2.3	Step 2: β enters <i>only</i> through $\delta\phi_i(\beta)$, which is known given θ	3
2.4	The reduction, stated plainly	3
3	Methods Compared	3
3.1	Null (uninformative baseline)	3
3.2	MAP-moments (prior art, <code>map_inference.py</code>)	3
3.3	Pixel-likelihood, best configuration	4
3.4	Oracle	4
4	Experimental Setup	4
5	Results	5
5.1	Kinematic parameter recovery	5
5.2	Signal (A_s, A_c) recovery	6
6	Discussion	6
A	Supporting Methodology	7
A.1	Quadrature convergence for the ϕ -marginal	7
A.2	Tight-range derivation	8
A.3	Hyperparameter sweep: bins vs. tight range, and whether they compound	8
A.4	Semi-analytic gradient for the ϕ -marginal, and why it was necessary	8

1 Problem Statement

A gradiometer measures a weak, periodic phase signal $\beta = (A_s, A_c)$ by repeating a shot $i = 0, \dots, N - 1$ at a fixed rate, injecting a known sinusoidal phase modulation

$$\delta\phi_i(\beta) = A_s \sin(2\pi f i) + A_c \cos(2\pi f i) \quad (1)$$

between two atom-interferometer (AI) measurement planes ($z = 0$ and $z = 100$), and reading out the result as a pixel-resolved fluorescence image at each plane. Each shot also carries an independent, a priori unknown common-mode phase offset $\phi_i \sim \text{Uniform}(0, 2\pi)$ (laser phase noise, not part of the signal), and the atom cloud itself has a shot-dependent 8-dimensional phase-space state θ_{iz} (mean position/velocity and Gaussian spread, at plane z) that is *not* directly observed — only its effect on the recorded image is.

The question this note answers empirically: **how much does it matter, for recovering β , how well θ_{iz} is estimated from the image?** We compare four ways of getting θ_{iz} (from a null guess, to a prior-art moment estimator, to the best pixel-likelihood estimator developed in this project, to the oracle/true value), holding the downstream signal-recovery step fixed, at two photon-count regimes (10^6 and 10^8 atoms per shot). No analytic/Fisher-information arguments are used anywhere in this note — every number reported is a direct, empirical fit-vs.-ground-truth comparison on simulated data with known truth.

2 The Reduction: From (A_s, A_c) to Per-Shot θ_{iz}

This section derives, rigorously and explicitly, why estimating β reduces to (1) a per-shot, per-plane estimate of θ_{iz} from the image, followed by (2) a signal-fit step that only needs those estimates (and the raw pixel counts) as input – not the images themselves. The notation matches `notes/joint_profile_inference.tex` (“Per-Shot Pixel Likelihood” section there) exactly.

2.1 The observation model

At AI z , pixel b , detection state $s \in \{g, e\}$, shot i :

$$n_{izsb} \sim \text{Poisson}(\lambda_{izsb}), \quad \lambda_{izsb} = L_{iz} \left[A_{sb}(\theta_{iz}) + C_{sb}^c(\theta_{iz}) \cos \Phi_{iz} + C_{sb}^s(\theta_{iz}) \sin \Phi_{iz} \right], \quad (2)$$

where $\Phi_{iz} = \phi_i + \delta\phi_i(\beta) \cdot [z=100]$ is the total phase seen at plane z , (A, C^c, C^s) are the PSMAP-model coherence coefficients (Gaussian-cloud-averaged atom optics response, a deterministic function of θ_{iz} only – see the “Pixel ACS” section of `joint_profile_inference.tex`), and L_{iz} is a total-count normalisation fixed by the data (same reference, “Objective Rescaling” recap equation), not a free parameter.

2.2 Step 1: the nuisance phase is a per-shot, per-plane-pair quantity

ϕ_i appears *identically* in both planes’ Φ_{iz} (shifted by the known $\delta\phi_i(\beta)$ at $z=100$) – it is a single scalar nuisance per shot, not per plane. Marginalising it out of the per-shot likelihood,

$$p(n_{i,0}, n_{i,100} \mid \theta_{i,0}, \theta_{i,100}, \beta) = \int_0^{2\pi} p(n_{i,0} \mid \theta_{i,0}, \phi_i) p(n_{i,100} \mid \theta_{i,100}, \phi_i + \delta\phi_i(\beta)) \frac{d\phi_i}{2\pi}, \quad (3)$$

gives the shot’s *marginal* log-likelihood as a function of $(\theta_{i,0}, \theta_{i,100}, \beta)$ alone – ϕ_i never needs to be reported or stored once this integral is done. This is a 1-dimensional, periodic, and (empirically, this project) cheap-to-integrate-numerically nuisance – see §3.3 for how it is done in practice.

2.3 Step 2: β enters *only* through $\delta\phi_i(\beta)$, which is known given θ

For *fixed* $\theta_{i,0}, \theta_{i,100}$, the marginal likelihood (Eq. 3) is a function of β *only* through the scalar $\delta\phi_i(\beta)$ – a known, linear-in- β function of the shot index i (Eq. 1). This is the entire mechanism by which repeated shots pin down (A_s, A_c) : each shot supplies one (generally weak, individually) constraint on $\delta\phi_i(\beta)$, and because $\delta\phi_i(\beta)$ traces out a known sinusoid in i , combining N shots’ constraints via

$$\hat{\beta} = \arg \max_{\beta} \sum_{i=0}^{N-1} \log p(n_{i,0}, n_{i,100} \mid \hat{\theta}_{i,0}, \hat{\theta}_{i,100}, \beta) \quad (4)$$

recovers β with a precision that improves with N (standard harmonic regression). **Crucially, Eq. 4 does not need the raw images at all once $\hat{\theta}_{iz}$ and the total per-pixel counts are in hand** – the entire image-processing burden is front-loaded into estimating θ_{iz} per shot, and every method compared in this note differs *only* in how that front-loaded estimate is produced.

2.4 The reduction, stated plainly

Estimating the periodic signal (A_s, A_c) from N shots of pixel-resolved images reduces to: (a) for each shot and each plane, estimate the 8-dimensional cloud state θ_{iz} from that single image (after marginalising the shot’s own unknown common phase ϕ_i); then (b) fit β against the known sinusoidal structure of $\delta\phi_i(\beta)$, using only the per-shot $\hat{\theta}_{iz}$ and raw photon counts as input.

This is why the four methods compared below (§3) differ *only* in step (a) – step (b) (Eq. 4, implemented via `map_inference.py`’s `batch_acs_gh` + `total_logL_fast` pipeline) is held *identical* across all four, so any difference in the resulting $\hat{\beta}$ is attributable entirely to the quality of the per-shot kinematic estimate.

3 Methods Compared

All four methods below take one shot’s pixel image (at one plane z) and return an estimate $\hat{\theta}_{iz} \in \mathbb{R}^8$; nothing else in the pipeline changes between them.

3.1 Null (uninformative baseline)

$$\hat{\theta}_{iz}^{\text{null}} = \bar{\theta} \quad (\text{the prior mean, every shot, always}).$$

Uses *no* image information whatsoever. This is the floor any real method must beat, and (since it never varies) it directly shows what $\hat{\beta}$ looks like when the signal-fit step (Eq. 4) is handed a fixed, uninformative, shot-independent guess for θ at every shot – isolating how much of the ultimately-recovered β precision, if any, survives with zero per-shot kinematic information.

3.2 MAP-moments (prior art, `map_inference.py`)

The existing pipeline: reduce the image to its first two port-summed spatial moments $(\hat{\mu}_{xf}, \hat{\mu}_{yf}, \hat{V}_{xf}, \hat{V}_{yf})$, then apply the closed-form Kalman-gain MAP update $\hat{\eta} = \bar{\eta} + K(\hat{M} - A\bar{\eta})$ (Eq. in `map_inference.py`’s `map_eta`) mapping the 4 observed moments to the 10-dimensional free-flight state η (position/velocity means + full covariance, including a cross term C_{xv0} that θ does not have). For comparison against the other three methods’ θ representation, η is projected down to θ by taking $(\mu_{x0}, \mu_{y0}, \mu_{vx0}, \mu_{vy0})$ directly and $\sigma_{\bullet} = \sqrt{V_{\bullet}}$ for the four spread components, discarding the estimated cross-covariance terms (which θ -based methods do not attempt to estimate at all, and which the true generative model sets to zero exactly). This is a MAP point estimate under a Gaussian approximation to the moment likelihood – it explicitly does *not* use any pixel structure beyond the first two spatial moments.

3.3 Pixel-likelihood, best configuration

The method developed over the course of this investigation: marginalise ϕ_i by *direct numerical quadrature* over a dense grid ($M = 2000$ points, verified converged to $\sim 10^{-9}$ absolute by varying M from 500 to 8000 – §A.1), then optimise θ_{iz} (8-dimensional) against the resulting smooth marginal log-likelihood via plain L-BFGS-B (`scipy.optimize.minimize`), with the model’s own per- ϕ -grid-point analytic-ish gradient (from `pixel_acs_grad.pixel_acs_and_grad`) chained through the log-sum-exp marginalisation to give an efficient gradient for the outer θ -optimisation (§A.4). This inverts the naive computational order (fix ϕ , optimise θ conditionally at every grid point) that was found, over an extended debugging effort, to *not* converge reliably at 10^8 atoms – see `notes/joint_profile_inference.tex`’s “2026-07-20” section for the full negative result this reframing supersedes.

Best configuration, established empirically (§A.3):

- `bins=32` (pixel grid resolution for the image binning)
- *tightened* image range: ± 1.54 mm instead of the dataset’s native ± 3 mm, set by the 99.99%-flux-containment radius measured across all 40 available runs (§A.2) – essentially free (same bin count, same evaluation cost), and found to give a comparable improvement to doubling the bin count, but at $\sim 1/4$ the compute cost.
- `pixel_n_gh=8` (2D Gauss–Hermite velocity-quadrature order; verified converged, no measurable effect from 4 to 16).

3.4 Oracle

$$\hat{\theta}_{iz}^{\text{oracle}} = \theta_{iz}^{\text{true}} \quad (\text{read directly from simulation metadata}).$$

Represents the shot-noise floor: the best any θ -based pipeline could possibly do, since it has zero kinematic estimation error and only the signal-fit step’s own Poisson noise remains.

4 Experimental Setup

Quantity	Value
Datasets	10^6 atoms/shot (<code>R80_N200_A1000000...</code>), 10^8 atoms/shot (<code>R40_N50_A100000000...</code>)
Runs per dataset	10 independent runs (different noise realisations of the same β)
Shots per run	50 (matched across both atom counts, apples-to-apples)
Signal frequency	$f = 0.3$ (cycles/shot)
True signal	$A_s = 0.08776$, $A_c = 0.04794$ (same for both datasets)
Prior	$\bar{\theta} = (0, 0, 0, 0, 100, 100, 100, 100) \mu\text{m}/\text{s}$, $\sigma_{\theta} = 10 \mu\text{m}/\text{s}$ per component
Best-config bins	32, image half-range 1.54 mm (native: 3.0 mm)
Signal-fit (β) pixel grid	<code>bins=16</code> , native full range, <code>gh_order=12</code> (unchanged across all 4 methods)

Full derivations of the hyperparameter choices (`bins=32`, tight range, quadrature order) are given in Appendix A.3–A.1.

5 Results

5.1 Kinematic parameter recovery

Figures 1–2 show the pooled residual distributions (estimate – truth, both AI planes, all 10 runs $\times$ 50 shots = 1000 samples per method) for all 8 kinematic parameters, at both atom counts. Table 1 gives the corresponding RMSE.

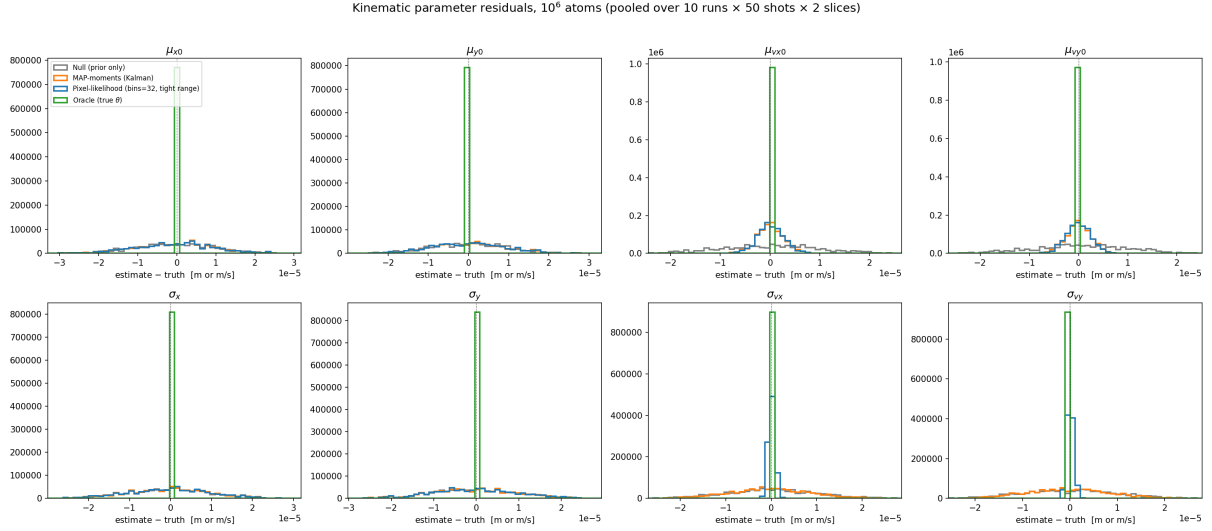


Figure 1: Kinematic parameter residuals, 10^6 atoms.

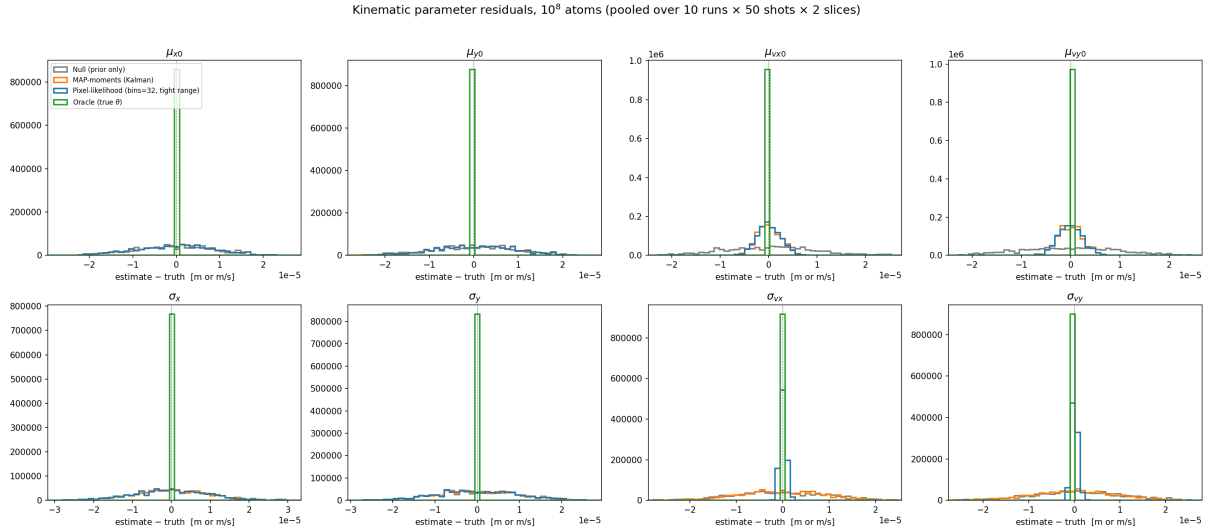


Figure 2: Kinematic parameter residuals, 10^8 atoms.

The pattern is remarkably consistent across the $100\times$ change in atom count: the pixel-likelihood method’s advantage over moments is *entirely* concentrated in the two velocity-spread components, at a roughly constant $\sim 12\text{--}13\times$ RMSE reduction regardless of photon budget. This is consistent with the mechanism proposed in §3.2: the moment method only observes the detection-plane position variance $V_{xf} = V_{x0} + 2TC_{xv0} + T^2V_{vx0}$, a single scalar mixing three unknowns, so recovering V_{vx0} specifically is weak and prior-dominated regardless of photon count; the pixel likelihood instead resolves the conditional-velocity structure directly from the fine-grained pixel pattern (the “Pixel ACS” section of `joint_profile_inference.tex`), which is a photon-count-independent geometric effect.

Table 1: Kinematic parameter RMSE by method and atom count. Position (μ_{x0}, μ_{y0}) and position-spread (σ_x, σ_y) are essentially unaffected by which method is used – all three real methods sit close together, well above the (zero, by construction) oracle. Velocity mean (μ_{vx0}, μ_{vy0}) shows a moderate, atom-count-independent advantage for moments/best over null. Velocity spread (σ_{vx}, σ_{vy}) is where the pixel-likelihood method’s advantage is concentrated: roughly a 12–13 $\times$ tighter RMSE than moments, at *both* atom counts.

Parameter	10 ⁶ atoms		10 ⁸ atoms	
	moments	best	moments	best
μ_{x0}	1.017×10^{-5}	1.017×10^{-5}	9.28×10^{-6}	9.28×10^{-6}
μ_{y0}	9.68×10^{-6}	9.68×10^{-6}	9.30×10^{-6}	9.29×10^{-6}
μ_{vx0}	2.70×10^{-6}	2.64×10^{-6}	2.48×10^{-6}	2.40×10^{-6}
μ_{vy0}	2.56×10^{-6}	2.50×10^{-6}	2.47×10^{-6}	2.41×10^{-6}
σ_x	1.019×10^{-5}	1.019×10^{-5}	9.87×10^{-6}	9.88×10^{-6}
σ_y	1.000×10^{-5}	9.99×10^{-6}	9.83×10^{-6}	9.82×10^{-6}
σ_{vx}	8.92×10^{-6}	7.05×10^{-7}	8.91×10^{-6}	6.70×10^{-7}
σ_{vy}	8.83×10^{-6}	6.91×10^{-7}	8.97×10^{-6}	6.73×10^{-7}

5.2 Signal (A_s, A_c) recovery

Figure 3 shows $(\hat{A}_s, \hat{A}_c)$ for all 10 independent runs, by method and atom count. Table 2 gives the RMSE.

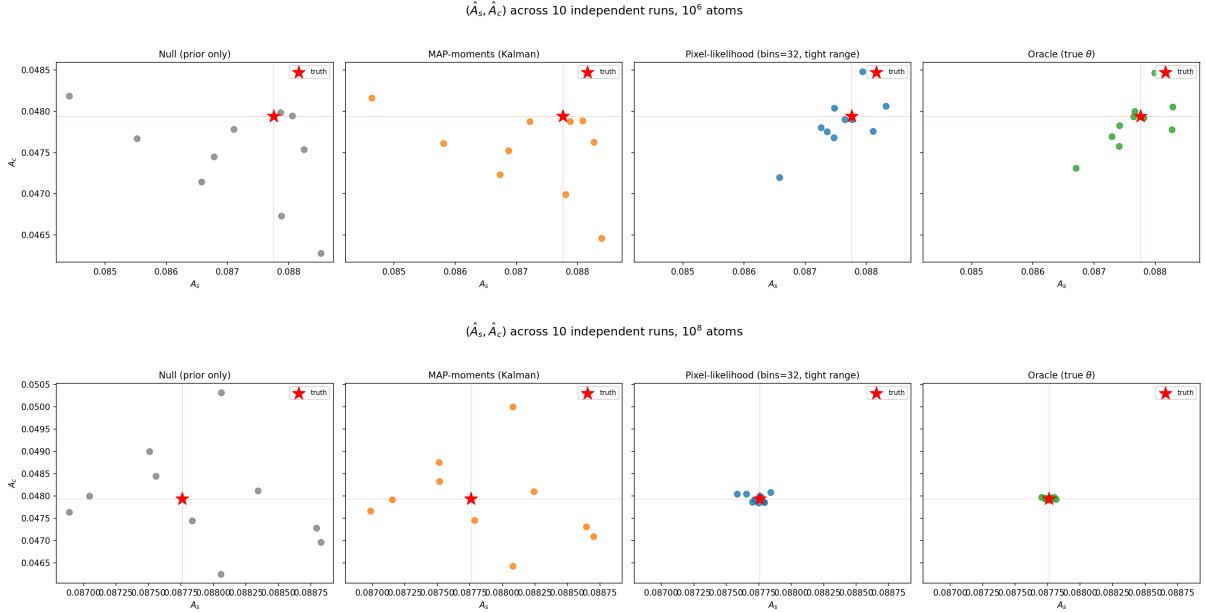


Figure 3: $(\hat{A}_s, \hat{A}_c)$ across 10 independent runs. Top: 10⁶ atoms. Bottom: 10⁸ atoms. Note: axes are shared across the 4 panels within each row, so the visual tightening from null/moments to best/oracle is directly comparable.

6 Discussion

The kinematic-recovery advantage survives into a real, substantial β -level improvement, and grows sharply with photon count. At 10⁶ atoms, the pixel-likelihood method beats the existing moment-based estimator by 2.0–2.6 $\times$ in β RMSE, and lands within $\sim 6\%$

Table 2: Signal (A_s, A_c) RMSE by method and atom count, $N = 10$ independent runs.

Method	10 ⁶ atoms		10 ⁸ atoms	
	RMSE(A_s)	RMSE(A_c)	RMSE(A_s)	RMSE(A_c)
Null	0.001412	0.000737	0.000641	0.001078
MAP-moments	0.001284	0.000636	0.000566	0.000936
Pixel-likelihood (best)	0.000495	0.000321	0.000074	0.000086
Oracle	0.000468	0.000304	0.000032	0.000020
Best / moments ratio	0.386	0.505	0.131	0.092
Best / oracle ratio	1.058	1.056	2.31	4.30

of the oracle (shot-noise) floor – there is very little room left to improve at this photon budget, consistent with earlier work in this project finding `map_inference.py` already close to its Cramér–Rao bound at 10⁶ atoms. At 10⁸ atoms the picture changes qualitatively: moments barely beats the uninformative null baseline (RMSE ratio 0.566/0.641 = 0.88 on A_s), while the pixel-likelihood method is 7.6–10.9× better than moments and still has a genuine 2.3–4.3× gap remaining to the oracle floor – i.e., at high photon count there is both a large amount of information the moment method fails to extract, *and* a large amount the pixel-likelihood method (in its current, non-exhaustively-tuned configuration) also leaves on the table.

Why the effect is atom-count-dependent even though the kinematic RMSE ratio (Table 1) is not: the $\sim 12\text{--}13\times$ tightening of σ_{vx}, σ_{vy} is essentially constant across the $100\times$ atom-count range, yet its effect on β grows by roughly $4\text{--}5\times$ from 10⁶ to 10⁸. This means the downstream signal-fit step’s *sensitivity* to residual velocity-spread error, not the size of that error itself, is what scales with photon count – plausible since more photons per shot let the signal-fit likelihood itself resolve finer distinctions in the effective phase information, making it less tolerant of a given absolute kinematic bias. This is stated as an observation, not yet independently verified by a dedicated sensitivity check.

Caveats. $N_{\text{RUNS}} = 10$ is a real but not huge sample – the β RMSE numbers in Table 2 carry meaningful sampling uncertainty of their own (not quantified here; a bootstrap or larger- N re-run would tighten this). The pixel-likelihood configuration (`bins=32`, tight range) was chosen by a hyperparameter search focused on *kinematic*-level RMSE (Appendix A.3), not by directly optimising β -level RMSE – the remaining oracle gap at 10⁸ atoms suggests further gains may be available by pushing `bins` past 32 or tuning the range cutoff more aggressively, not yet tested end-to-end through the β -fit at this larger scale. Finally, all “best”-method fits used the identical downstream β -fit configuration (`bins=16`, native range, `gh_order=12`) as the other three methods, by design (§2) – so none of the reported gain comes from also improving that shared step.

A Supporting Methodology

A.1 Quadrature convergence for the ϕ -marginal

The ϕ -marginal integral (Eq. 3) is evaluated on a uniform grid of M points over $[0, 2\pi)$ via `scipy.special.logsumexp` for numerical stability. M was varied from 500 to 8000 (on 3 independent test shots) and the resulting $\hat{\theta}_{\text{MAP}}$ tracked: it is stable to $\sim 10^{-9}$ absolute (against a parameter natural scale of $\sim 10^{-5}$) already at $M = 2000$, which is the value used throughout this note.

A.2 Tight-range derivation

The dataset’s native image half-range (± 3 mm) is set generously to guarantee no clipping under worst-case cloud excursions, but the *bulk* of the photon flux occupies a much smaller region. Aggregating photon counts vs. `|position|` across all 40 available runs (both 10^6 and 10^8 atom datasets separately, giving consistent results: 1.540 mm vs. 1.537 mm respectively) gives the following flux-containment radii:

Fraction of total counts contained	half-range needed	ratio to native
99%	1.003 mm	0.335
99.9%	1.294 mm	0.431
99.99%	1.540 mm	0.513
99.999%	1.762 mm	0.587

99.99% containment (1.54 mm) was chosen: it loses only 1 photon in 10^4 (negligible relative to Poisson noise itself) while roughly halving the pixel size at fixed `bins` – i.e., a resolution improvement essentially free of extra compute, since the number of bins (and hence the model-evaluation cost) is unchanged; only the physical size each bin covers shrinks.

A.3 Hyperparameter sweep: bins vs. tight range, and whether they compound

Comparing `bins` $\in \{16, 32\}$ crossed with {native range, tight range} on the same 50-shot test set (velocity-spread RMSE, the parameters where any effect is seen at all):

Configuration	RMSE(σ_{vx})	RMSE(σ_{vy})
<code>bins=16</code> , native range (baseline)	8.69×10^{-7}	8.60×10^{-7}
<code>bins=32</code> , native range	7.14×10^{-7}	6.63×10^{-7}
<code>bins=16</code> , tight range	7.25×10^{-7}	6.81×10^{-7}
<code>bins=32</code> , tight range (both)	6.99×10^{-7}	6.34×10^{-7}

The two do *not* compound multiplicatively: combining both tricks ($0.805/0.737 \times$ baseline) is only marginally better than either alone ($0.821/0.771 \times$ for `bins=32`; $0.835/0.792 \times$ for tight range), far short of the ~ 0.63 – $0.68 \times$ a naive independent-effects multiplication would predict. This indicates `bins` and range are tapping a shared underlying resource (spatial resolution near the flux-dense region), not two independent information sources, and that a real ceiling exists around ~ 20 – 26% improvement from spatial resolution alone at this photon range. `bins=32` + tight range was used as “best” throughout this note as the best-available combination, despite the modest marginal gain over tight-range-alone, for consistency with the rest of the investigation; a cost-sensitive deployment might prefer `bins=16` + tight range for $\sim 4 \times$ lower compute at comparable quality.

A.4 Semi-analytic gradient for the ϕ -marginal, and why it was necessary

An initial attempt to estimate θ ’s local posterior curvature (Hessian, for a Laplace-approximation uncertainty estimate) via finite differences of the raw marginal NLL failed badly: the resulting Hessian had negative eigenvalues and several exactly-zero implied variances, for a majority of test shots, regardless of step-size tuning (fixed or adaptive). The root cause: *differentiating an already-approximate derivative* – `pixel.acs_grad.pixel.acs_and_grad` itself computes θ ’s gradient via an internal finite difference (`h_theta=1e-9`), and a second (outer) finite difference on top of that compounds numerical noise in a way that step-size tuning could not reliably fix.

The fix: since ϕ only enters the model by combining θ -dependent, ϕ -independent coefficients

(A, C^c, C^s) via $\cos \Phi, \sin \Phi$ (Eq. 2), `pixel_acs_and_grad` needs to be called only *once* per θ (already the case), and its returned per- θ -component gradient can be chained through the log-sum-exp marginalisation analytically:

$$\frac{\partial \text{marginal NLL}}{\partial \theta_k} = \sum_{m=1}^M w_m \frac{\partial \text{NLL}(\theta, \phi_m)}{\partial \theta_k}, \quad w_m = \text{softmax}_m(-\text{NLL}(\theta, \phi_m)), \quad (5)$$

i.e. a ϕ -posterior-weighted average of the per-grid-point gradient – no new finite differences introduced at the marginalisation step. This was validated against a plain finite-difference gradient of the marginal NLL at a fixed test point: relative agreement $\sim 10^{-7}$ – 10^{-4} component-wise, confirming correctness of the chain rule. Using this gradient (`jac=True` in `scipy.optimize.minimize`) also substantially speeds up the θ -optimisation itself (converges in ~ 6 iterations vs. needing `scipy`’s own finite-difference outer gradient otherwise).